

PRÁCTICA N.º 4 — ENUNCIADO

GraphQL y el patrón Backend for Frontend

Un esquema que actúa como contrato, resolvers que componen datos de más de un servicio y una comparación medida contra REST

Asignatura	COM-600 · Microservicios · Gestión 2/2026
Tema del programa	Tema 4 — Comunicación entre Microservicios
Modalidad	Parte 1 guiada en laboratorio · Parte 2 individual
Duración estimada	Parte 1: 4 horas · Parte 2: trabajo autónomo
Puntaje	100 puntos (40 + 60)
Carpeta de trabajo	practica4-graphql

Nota: Este documento es el ENUNCIADO: contiene los comandos, las explicaciones y los ejercicios. Las capturas y las respuestas NO se escriben aquí, sino en el documento complementario «Hoja de Evidencias», que es el que se entrega.

1. Objetivos

Al finalizar esta práctica el estudiante será capaz de:

- Escribir un esquema GraphQL y leerlo como lo que es: el contrato público entre el cliente y el servidor.
- Implementar resolvers que obtengan los datos de una base MySQL, con argumentos y con variables.
- Diseñar mutaciones que devuelvan el estado resultante y no solo un mensaje de confirmación.
- Componer en un solo campo datos que viven en dos servicios distintos, reconociendo ahí el patrón Backend for Frontend.
- Detectar el problema N+1 con registros de consultas y resolverlo con carga por lotes.
- Manejar el fallo de un resolver y proteger el servidor con límites de profundidad y de complejidad.
- Comparar REST y GraphQL con datos medidos —viajes de red y tamaño de la carga— y no con opiniones.

2. Qué necesita saber antes de empezar

El contenido conceptual de esta práctica se desarrolla en **Tema 4 — Comunicación entre Microservicios**. Aquí solo se recuerda lo mínimo necesario para ejecutar los laboratorios.

- **Esquema:** el archivo que declara qué se puede pedir y con qué forma. Es el contrato: si no está en el esquema, no existe para el cliente.
- **Tipo:** la forma de un dato del esquema. El signo de admiración marca lo obligatorio y los corchetes marcan una lista.
- **Resolver:** la función que responde por UN campo del esquema. GraphQL decide a cuáles llamar según lo que el cliente pidió.
- **Consulta y mutación:** consulta es la operación de solo lectura; mutación es la que cambia el estado. Se separan por convención del lenguaje, no por el método HTTP.
- **N+1:** una consulta que trae N elementos y dispara una consulta adicional por cada uno. La respuesta sale bien y el servidor sufre en silencio.

Repaso: La discusión conceptual —estilos de comunicación entre microservicios, sobreobtención y subobtención, cuándo conviene un agregador y cuándo no— está en las diapositivas del Tema 4. Esta práctica es la ejecución de esas ideas sobre un servicio que usted ya tiene funcionando.

3. Requisitos previos

Requisito	Verificación	Observación
Node.js 20 o superior	node --version	La 20 o la 22 LTS: se usa el fetch global, que no existe en versiones anteriores
npm 10 o superior	npm --version	Viene con Node
MySQL 8 en ejecución	mysql --version	Sirve el servidor local, XAMPP o un contenedor de la Práctica 2
Servicio REST de la Práctica 3	curl http://localhost:3000/usuarios	Sigue existiendo y debe quedar corriendo: en el Laboratorio 4 se le pone delante la capa GraphQL
Base de datos propia (alternativa)	Su esquema de la Práctica 1 o 3	Puede trabajar sobre su propia base en lugar de practica4_ventas; conserve sus nombres de tabla y ajuste los SELECT
curl o Postman	curl --version	Necesario para las mediciones del Laboratorio 7
Git	git --version	Para subir la entrega al repositorio

Atención: Cree una carpeta de trabajo llamada practica4-graphql y realice TODO el trabajo dentro de ella. Esa carpeta se sube al repositorio al final.

Los comandos se muestran precedidos por el símbolo \$. NO escriba el símbolo \$ al copiar el comando.

El servicio REST de la Práctica 3 NO se reescribe ni se reemplaza: debe quedar corriendo en su puerto durante los laboratorios 4 al 7. Esta práctica le pone una capa delante, no lo sustituye.

4. Estructura y forma de entrega

La práctica se apoya en dos documentos:

Documento	Para qué sirve	¿Se entrega?
Enunciado (este archivo)	Guía de trabajo: comandos, explicaciones, ejercicios y rúbrica. Se consulta mientras se trabaja.	No
Hoja de Evidencias	Formulario donde se pegan las 19 capturas y se responden las 8 preguntas de comprobación de la Parte 1.	Sí, en PDF

El trabajo tiene dos partes obligatorias:

Parte	Contenido	Puntaje
PARTE 1 — Guiada	8 laboratorios paso a paso. Se ejecutan los comandos y se documenta cada resultado en la Hoja de Evidencias.	40 pts
PARTE 2 — Ejercicios propuestos	6 ejercicios de resolución individual. Se entregan como código en el repositorio más un informe en PDF.	60 pts
TOTAL		100 pts

Entregables

- Hoja de Evidencias en PDF, con las capturas pegadas y las preguntas de comprobación respondidas (Parte 1).
- Repositorio Git público con la carpeta practica4-graphql: src/ con el esquema, los resolvers y los cargadores, el script .sql de la base y un README.md con las instrucciones para levantar los dos servicios.
- Informe PDF de la Parte 2 con la resolución de cada ejercicio, las capturas que la demuestren y la tabla de medición del ejercicio 6.

PARTE 1 — PRÁCTICA GUIADA

Ejecute cada comando en el orden indicado. Cuando aparezca un aviso de «Captura N.º», tome la captura de pantalla y péguela en el recuadro correspondiente de la Hoja de Evidencias. Las «Pregunta N.º» se responden en esa misma hoja, con sus propias palabras: son las que demuestran que entendió lo que ejecutó.

Consejo: Trabaje con los dos documentos abiertos a la vez: este enunciado para leer y ejecutar, y la Hoja de Evidencias para ir registrando. Las capturas y las preguntas están numeradas igual en ambos.

Laboratorio 0 — Entorno y proyecto base

Antes de escribir una línea de esquema, confirme que las tres piezas están en su lugar: Node, MySQL y el servicio REST de la Práctica 3, que sigue vivo y va a seguir estándolo.

Paso 1. Verifique las versiones y el servicio que ya tiene

Abra una terminal y ejecute los cuatro comandos. El último debe responder con el JSON de los usuarios de la Práctica 3: es el servicio sobre el que después vamos a componer.

```
$ node --version
$ npm --version
$ mysql --version
$ curl http://localhost:3000/usuarios
```

Si el último comando responde «Connection refused», levante el servicio de la Práctica 3 antes de continuar. Si no conserva ese proyecto, el Laboratorio 4 indica el mínimo que debe exponer para que esta práctica funcione.

Captura N.º 1: salida de los cuatro comandos anteriores, con el JSON de usuarios visible. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 1: El servicio REST de la Práctica 3 sigue funcionando exactamente igual que antes y esta práctica no le cambia una sola línea. ¿Qué gana un equipo cuando la capa nueva se pone delante en lugar de reescribir el servicio existente? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Paso 2. Cree el proyecto e instale las dependencias

Trabajaremos con Apollo Server sobre Node. Si prefiere graphql-yoga, puede usarlo: los esquemas y los resolvers de esta práctica son idénticos, solo cambia el arranque del servidor.

```
$ mkdir practica4-graphql && cd practica4-graphql
$ npm init -y
$ npm pkg set type=commonjs
$ npm install @apollo/server graphql mysql2 dataloader
$ npm install graphql-depth-limit
# @apollo/server: servidor · graphql: el lenguaje
# mysql2: cliente de la base · dataloader: carga por lotes (Laboratorio 5)
```

Paso 3. Cree la base de datos

Entre a MySQL con su usuario y ejecute el script. Note que ventas guarda cliente_id y NO el nombre del cliente: el nombre vive en el servicio REST de la Práctica 3 y lo iremos a buscar allá en el Laboratorio 4.

```
db/practica4.sql
CREATE DATABASE IF NOT EXISTS practica4_ventas;
```

```
USE practica4_ventas;

CREATE TABLE ventas (
  id INT AUTO_INCREMENT PRIMARY KEY,
  cliente_id INT NOT NULL,
  fecha DATE NOT NULL,
  total DECIMAL(10,2) DEFAULT 0
);

CREATE TABLE detalle_venta (
  id INT AUTO_INCREMENT PRIMARY KEY,
  venta_id INT NOT NULL,
  producto VARCHAR(100) NOT NULL,
  cantidad INT NOT NULL,
  precio_unitario DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (venta_id) REFERENCES ventas(id) ON DELETE CASCADE
);
```

Si usa su propia base: Puede trabajar sobre el esquema que ya tiene de la Práctica 1 o de la Práctica 3. Conserve sus nombres de tabla y de columna, y ajuste los SELECT de los resolvers. Lo único que esta práctica necesita es una entidad principal, una entidad hija en relación de uno a muchos y una clave que apunte a un dato que viva en el otro servicio.

Paso 4. Cargue los datos de prueba

Los valores de cliente_id deben corresponder a usuarios que existan en el servicio REST de la Práctica 3. Verifique con la salida del primer paso antes de insertar.

```
db/practica4.sql (continuación)

INSERT INTO ventas (cliente_id, fecha, total) VALUES
  (1, '2026-08-03', 450.00), (2, '2026-08-04', 120.50),
  (1, '2026-08-05', 280.00);

INSERT INTO detalle_venta (venta_id, producto, cantidad,
  precio_unitario) VALUES
  (1, 'Laptop', 1, 350.00), (1, 'Mouse', 2, 50.00),
  (2, 'Teclado', 1, 120.50), (3, 'Monitor', 1, 280.00);
```

Compruebe con `SELECT * FROM ventas;` que las tres filas están cargadas.

Captura N.º 2: consola de MySQL mostrando el resultado de `SELECT * FROM ventas;` con las tres filas insertadas. → *péguela en la Hoja de Evidencias.*

Paso 5. Escriba el acceso a la base con contador de consultas

Este archivo hace dos cosas: abre el grupo de conexiones y deja registro de CADA sentencia que sale hacia MySQL. Ese registro es la herramienta con la que en el Laboratorio 5 se ve el problema N+1 sin adivinar nada.

```
src/db.js

const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: 'localhost', user: 'root', password: '',
  database: 'practica4_ventas', connectionLimit: 10,
});

let n = 0;
async function q(sql, params = []) {
  n++;
  console.log('[SQL %d] %s', n, sql.replace(/\s+/g, ' ').trim());
```

```
const [filas] = await pool.query(sql, params);
return filas;
}

const contador = { leer: () => n, reiniciar: () => { n = 0; } };
module.exports = { q, contador };
```

Laboratorio 1 — El esquema como contrato

El esquema es lo primero que se escribe y lo último que se cambia sin avisar. Declara qué existe, con qué forma y qué es obligatorio. Todavía no hay datos reales: el objetivo de este laboratorio es que el contrato quede escrito y que el explorador lo entienda.

Paso 1. Escriba los tipos y la consulta

Cree la carpeta `src/` y dentro el archivo `esquema.js`. Lea con atención los signos: `Int!` es un entero obligatorio; `[DetalleVenta]!` es una lista obligatoria de elementos obligatorios; `String` sin admiración admite el valor nulo.

```
src/esquema.js
const typeDefs = `#graphql
  type DetalleVenta {
    id: Int!
    producto: String!
    cantidad: Int!
    precioUnitario: Float!
  }
  type Venta {
    id: Int!
    fecha: String!
    total: Float!
    detalle: [DetalleVenta]!
  }
  type Query {
    ventas: [Venta]!
    venta(id: Int!): Venta
  }
`;
```

Agregue al final del archivo la línea `module.exports = typeDefs`; Note que `venta(id: Int!)` devuelve `Venta` SIN admiración: si el identificador no existe, la respuesta legítima es nulo.

Paso 2. Escriba resolvers provisionales y levante el servidor

Para probar el contrato todavía no hace falta la base. Devolvemos datos fijos: así queda claro que el esquema y el origen de los datos son dos cosas separadas.

```
src/servidor.js
const { ApolloServer } = require('@apollo/server');
const { startStandaloneServer } = require('@apollo/server/standalone');
const typeDefs = require('./esquema');

const resolvers = {
  Query: {
    ventas: () => [{ id: 1, fecha: '2026-08-03', total: 450, detalle: [] }],
    venta: () => null,
  },
};

async function main() {
  const server = new ApolloServer({ typeDefs, resolvers });
  const { url } = await startStandaloneServer(server, {
    listen: { port: 4000 },
  });
}
```

```
});  
console.log('GraphQL escuchando en ' + url);  
}
```

Agregue `main();` al final del archivo y ejecute `node src/servidor.js`. El servidor debe anunciar `http://localhost:4000/`.

Paso 3. Abra el explorador y lea el contrato

Entre a `http://localhost:4000/` desde el navegador. Abra el panel de documentación del explorador: ahí está el esquema que usted acaba de escribir, sin que nadie lo haya documentado a mano.

Captura N.º 3: panel de documentación del explorador mostrando el tipo `Venta` con sus campos y sus obligaciones. → *péguela en la Hoja de Evidencias*.

Paso 4. Ejecute la primera consulta y provoque un error de contrato

Escriba y ejecute la primera consulta. Después agregue un campo que NO está en el esquema, por ejemplo `descuento`, y vuelva a ejecutar.

```
query {  
  ventas {  
    id  
    fecha  
    total  
  }  
}
```

Al pedir un campo inexistente, el servidor responde antes de tocar dato alguno: «Cannot query field "descuento" on type "Venta"». Esa validación ocurre contra el esquema, no contra la base.

Captura N.º 4: las dos ejecuciones: la consulta válida con su respuesta y el error `Cannot query field` al pedir un campo inexistente. → *péguela en la Hoja de Evidencias*.

Pregunta N.º 2: En REST, pedirle a un endpoint un campo que no existe suele devolver un 200 con ese campo ausente, y el error aparece recién en el cliente. Aquí el servidor lo rechaza antes de ejecutar nada. ¿Qué implica esa diferencia para un equipo con varios consumidores del mismo servicio? → *respóndala en la Hoja de Evidencias, con sus propias palabras*.

Laboratorio 2 — Resolvers: de dónde salen los datos

El esquema dice qué existe; el resolver dice de dónde sale. Ahora reemplazamos los datos fijos por la base MySQL y agregamos argumentos y variables.

Paso 1. Escriba los resolvers contra MySQL

Cree `src/resolvers.js`. Observe el alias `precio_unitario` AS `precioUnitario`: la base usa guiones bajos y el esquema usa mayúscula intercalada. El resolver es exactamente el lugar donde se traduce una convención a la otra.

```
src/resolvers.js  
const { q } = require('./db');  
  
const resolvers = {
```

```
Query: {
  ventas: () => q('SELECT id, fecha, total FROM ventas ORDER BY id'),
  venta: (_p, { id }) =>
    q('SELECT id, fecha, total FROM ventas WHERE id = ?', [id])
    .then(f => f[0] || null),
},
Venta: {
  detalle: (venta) => q(
    'SELECT id, producto, cantidad, precio_unitario AS precioUnitario '
    + 'FROM detalle_venta WHERE venta_id = ?', [venta.id]),
},
};

module.exports = resolvers;
```

En `src/servidor.js` borre el objeto `resolvers` provisional y reemplácelo por `const resolvers = require("./resolvers");`. Reinicie el servidor.

Los cuatro parámetros de un resolver: Un resolver recibe (padre, argumentos, contexto, información). El primero es el objeto del que cuelga el campo: en `Venta.detalle`, el padre es la venta, y por eso `venta.id` está disponible. El segundo trae los argumentos declarados en el esquema. El tercero es el contexto compartido de la petición, que usaremos en el Laboratorio 5.

Paso 2. Consulte con argumento

Ejecute esta consulta en el explorador. El argumento va escrito directamente dentro de la consulta.

```
query {
  venta(id: 1) {
    fecha
    total
    detalle {
      producto
      cantidad
      precioUnitario
    }
  }
}
```

Mire la terminal del servidor: verá dos sentencias SQL registradas, una por la venta y otra por su detalle. Recuérdelo, porque en el Laboratorio 5 ese número va a crecer.

Captura N.º 5: la consulta con argumento y su respuesta anidada en el explorador, junto a las sentencias SQL en la terminal del servidor. → *péguela en la Hoja de Evidencias*.

Paso 3. Consulte con variables

Escribir el valor dentro de la consulta sirve para probar, no para una aplicación real. Con variables, la consulta es una plantilla fija y solo cambian los datos. Escriba la consulta arriba y el JSON en el panel de variables del explorador.

```
query ObtenerVenta($id: Int!) {
  venta(id: $id) {
    fecha
    total
  }
}

# Panel de variables del explorador:
```

```
{ "id": 2 }
```

Cambie el valor de la variable a 99 y vuelva a ejecutar: la respuesta trae venta con valor nulo, sin error. Eso es exactamente lo que el esquema prometió al declarar Venta sin admiración.

Captura N.º 6: la consulta con variables ejecutada dos veces: con id 2 devolviendo datos y con id 99 devolviendo nulo. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 3: Con variables, la consulta enviada al servidor es siempre la misma cadena de texto y solo cambia el JSON de variables. Además del orden, ¿qué otro problema evita esa separación cuando el valor viene escrito por un usuario? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Laboratorio 3 — Mutaciones: cambiar el estado y devolver algo útil

Una mutación mal diseñada responde «ok». Una mutación bien diseñada devuelve el objeto resultante, para que el cliente pinte la pantalla sin tener que preguntar otra vez.

Paso 1. Declare las entradas y las mutaciones en el esquema

Agregue este bloque a src/esquema.js, dentro de las comillas invertidas. Un input es un tipo que solo sirve para entrar: no se puede devolver.

```
src/esquema.js (agregar dentro del esquema)
input DetalleInput {
  producto: String!
  cantidad: Int!
  precioUnitario: Float!
}
input VentaInput {
  clienteId: Int!
  fecha: String!
  detalle: [DetalleInput!]!
}
type Mutation {
  crearVenta(input: VentaInput!): Venta!
  cambiarCantidad(detalleId: Int!, cantidad: Int!): DetalleVenta!
}
```

Por qué devuelven el objeto: crearVenta devuelve Venta! y cambiarCantidad devuelve DetalleVenta!. El cliente decide en la misma llamada qué campos del resultado quiere. Si la mutación devolviera solo un texto, el cliente necesitaría un segundo viaje para refrescar la pantalla: justo lo que vinimos a evitar.

Paso 2. Implemente crearVenta

Agregue esta sección al objeto resolvers de src/resolvers.js, al mismo nivel que Query. El total se calcula en el servidor a partir del detalle recibido: nunca se confía en un total enviado por el cliente.

```
src/resolvers.js (agregar dentro de resolvers)
Mutation: {
  crearVenta: async (_, { input }) => {
    const total = input.detalle.reduce(
      (s, d) => s + d.cantidad * d.precioUnitario, 0);
    const r = await q(
      'INSERT INTO ventas (cliente_id, fecha, total) VALUES (?, ?, ?)',
      [input.clienteId, input.fecha, total]);
    for (const d of input.detalle) {
      await q('INSERT INTO detalle_venta (venta_id, producto, '
        + 'cantidad, precio_unitario) VALUES (?, ?, ?, ?)',
```

```
    [r.insertId, d.producto, d.cantidad, d.precioUnitario]);  
  }  
  return { id: r.insertId, fecha: input.fecha, total };  
},  
},  
},
```

Paso 3. Ejecute la mutación y pida el resultado

Note que la mutación pide de vuelta el detalle: ese campo lo resuelve Venta.detalle leyendo lo que se acaba de insertar.

```
mutation {  
  crearVenta(input: {  
    clienteId: 2  
    fecha: "2026-08-20"  
    detalle: [  
      { producto: "Impresora", cantidad: 1, precioUnitario: 620.00 }  
      { producto: "Cartucho", cantidad: 3, precioUnitario: 85.50 }  
    ]  
  }) {  
    id  
    total  
    detalle { producto cantidad }  
  }  
}
```

El total debe salir en 876.50 sin que usted lo haya enviado. Verifique en MySQL con `SELECT * FROM ventas ORDER BY id DESC LIMIT 1;`

Captura N.º 7: la mutación `crearVenta` ejecutada, con la respuesta que trae el identificador nuevo, el total calculado y el detalle insertado. → *péguela en la Hoja de Evidencias.*

Paso 4. Implemente `cambiarCantidad` y provoque el rechazo

Agregue esta mutación junto a la anterior. Después ejecútela con cantidad 0 para ver cómo responde GraphQL cuando un resolver lanza una excepción.

```
src/resolvers.js (agregar dentro de Mutation)  
  
cambiarCantidad: async (_p, { detalleId, cantidad }) => {  
  if (cantidad <= 0) {  
    throw new Error('La cantidad debe ser mayor que cero');  
  }  
  await q('UPDATE detalle_venta SET cantidad = ? WHERE id = ?',  
    [cantidad, detalleId]);  
  const filas = await q(  
    'SELECT id, producto, cantidad, precio_unitario AS '  
    + 'precioUnitario FROM detalle_venta WHERE id = ?', [detalleId]);  
  if (!filas[0]) throw new Error('No existe el ítem ' + detalleId);  
  return filas[0];  
},
```

Ejecute `cambiarCantidad(detalleId: 1, cantidad: 0)`. Observe que la respuesta HTTP sigue siendo 200 y que el mensaje viaja dentro de un arreglo llamado `errors`. Eso volverá en el Laboratorio 6.

Captura N.º 8: la mutación rechazada, mostrando el arreglo `errors` con el mensaje «La cantidad debe ser mayor que cero». → *péguela en la Hoja de Evidencias.*

Pregunta N.º 4: Compare esta respuesta con la que daría su servicio REST de la Práctica 3 ante la misma situación. ¿Qué pierde un cliente que solo mira el código de estado HTTP cuando habla con un servidor GraphQL? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Laboratorio 4 — Composición: aquí aparece el Backend for Frontend

Hasta ahora todo salió de una sola base. Ahora el campo cliente de una venta va a salir del servicio REST de la Práctica 3, que sigue corriendo intacto en su puerto 3000. La capa GraphQL no reemplaza a ese servicio: se pone delante, junta lo que la pantalla necesita y lo entrega en una sola respuesta. Eso es un Backend for Frontend.

Paso 1. Confirme que el servicio REST está arriba

Deje el servicio de la Práctica 3 corriendo en una terminal aparte. Si no conserva ese proyecto, basta con que exponga GET /usuarios y GET /usuarios/:id devolviendo un objeto con id, nombre y email.

```
$ curl -s http://localhost:3000/usuarios/1
$ curl -s http://localhost:3000/usuarios/2
# ambos deben devolver un objeto con id, nombre y email
```

Captura N.º 9: las dos respuestas del servicio REST de la Práctica 3, con los usuarios 1 y 2. → *péguela en la Hoja de Evidencias.*

Paso 2. Agregue el tipo Cliente y el campo compuesto al esquema

El cliente que consume este esquema no tiene forma de saber que ese dato viaja por HTTP hacia otro servicio. Para él es un campo más de Venta: esa es exactamente la ventaja y también el riesgo.

```
src/esquema.js (agregar)
type Cliente {
  id: Int!
  nombre: String!
  email: String
}

# dentro del type Venta que ya escribió, agregue:
#   clienteId: Int!
#   cliente: Cliente
```

Paso 3. Escriba el resolver que llama al servicio REST

Agregue este campo dentro de la sección Venta de src/resolvers.js, junto a detalle. Es el resolver que compone: recibe la venta que ya salió de MySQL y va a buscar el resto a otro servicio.

```
src/resolvers.js (agregar dentro de Venta)
clienteId: (venta) => venta.cliente_id,
cliente: async (venta) => {
  const base = process.env.URL_USUARIOS || 'http://localhost:3000';
  const r = await fetch(base + '/usuarios/' + venta.cliente_id);
  if (r.status === 404) return null;
  if (!r.ok) {
    throw new Error('Usuarios respondió ' + r.status);
  }
  return r.json();
},
```

Para que venta.cliente_id llegue al resolver, agregue cliente_id a los SELECT de Query.ventas y Query.venta. Reinicie el servidor.

Por qué la dirección se lee del entorno: En la Práctica 2 los servicios se llaman por el nombre del servicio dentro de la red de Docker y no por localhost. Leer la dirección de una variable de entorno permite que el mismo código funcione en su máquina y dentro de un compose sin tocar una línea.

Paso 4. Pida la pantalla completa en una sola consulta

Esta es la consulta que resuelve una pantalla real: encabezado de la venta, nombre del cliente y detalle. Un dato viene de MySQL, otro de otro servicio, y el cliente ve una sola respuesta.

```
query {  
  venta(id: 1) {  
    fecha  
    total  
    cliente {  
      nombre  
      email  
    }  
    detalle {  
      producto  
      cantidad  
    }  
  }  
}
```

Captura N.º 10: la consulta compuesta y su respuesta, con el nombre del cliente traído del servicio REST dentro del mismo objeto venta. → *péguela en la Hoja de Evidencias*.

Paso 5. Apague el servicio REST y vuelva a consultar

Detenga el servicio de la Práctica 3 con Control+C y ejecute otra vez la consulta anterior. Después vuelva a levantarlo.

La respuesta trae data con la venta y su detalle, y a la vez un arreglo errors por el campo cliente. La composición falló solo en la parte que dependía del otro servicio: el resto llegó igual.

Captura N.º 11: la respuesta parcial: data con los campos que sí se resolvieron y errors indicando el fallo del campo cliente. → *péguela en la Hoja de Evidencias*.

Pregunta N.º 5: Su capa GraphQL ahora depende de que el servicio REST esté vivo. Nombre dos consecuencias concretas de esa dependencia y una medida que usted tomaría para que la caída del servicio de usuarios no arrastre a toda la pantalla. → *respóndala en la Hoja de Evidencias, con sus propias palabras*.

Laboratorio 5 — El problema N+1 y la carga por lotes

La consulta responde bien, el cliente está contento y el servidor está haciendo un trabajo absurdo. El N+1 no se manifiesta como error: se manifiesta como lentitud sin causa aparente. Por eso hay que verlo, y para eso pusimos el contador de consultas en el Laboratorio 0.

Paso 1. Provoque el N+1 y cuéntelo

Limpie la terminal del servidor y ejecute esta consulta, que pide TODAS las ventas con su detalle.

```
query {  
  ventas {
```

```
    id
    detalle {
      producto
    }
  }
}
```

Cuenta las líneas [SQL n] en la terminal: una para traer las ventas y una más por cada venta. Con 4 ventas son 5 consultas; con 200 ventas serían 201. La respuesta es correcta en los dos casos.

Captura N.º 12: terminal del servidor mostrando la sentencia de ventas seguida de una sentencia de detalle_venta por cada venta. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 6: La consulta responde bien y ningún registro de errores se queja. ¿Por qué este problema suele aparecer recién en producción y no en la máquina del desarrollador? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Paso 2. Escriba el cargador por lotes

Cree src/cargadores.js. DataLoader junta todas las claves que los resolvers hermanos piden en el mismo tic del bucle de eventos y hace UNA sola consulta con IN. Después reparte el resultado en el mismo orden en que le pidieron las claves.

```
src/cargadores.js

const DataLoader = require('dataloader');
const { q } = require('./db');

function crearCargadores() {
  const detallePorVenta = new DataLoader(async (ids) => {
    const filas = await q(
      'SELECT id, venta_id, producto, cantidad, '
      + 'precio_unitario AS precioUnitario '
      + 'FROM detalle_venta WHERE venta_id IN (?)', [ids]);
    return ids.map(id => filas.filter(f => f.venta_id === id));
  });
  return { detallePorVenta };
}

module.exports = { crearCargadores };
```

Un juego de cargadores por petición: Los cargadores se crean para CADA petición y se tiran al terminarla. Si se crearan una sola vez al arrancar el servidor, la memoria interna de DataLoader devolvería datos viejos a peticiones posteriores: un error de caché difícil de rastrear.

Paso 3. Póngalo en el contexto y úselo en el resolver

El contexto es el tercer parámetro de todo resolver y es el lugar previsto para lo que dura una petición. Modifique el arranque del servidor y el resolver de detalle.

```
src/servidor.js y src/resolvers.js

// en src/servidor.js, dentro de startStandaloneServer:
listen: { port: 4000 },
context: async () => ({ cargadores: crearCargadores() }),

// en src/resolvers.js, reemplace Venta.detalle por:
detalle: (venta, _a, { cargadores }) =>
  cargadores.detallePorVenta.load(venta.id),
```

No olvide requerir crearCargadores en src/servidor.js. Reinicie y repita la consulta del primer paso.

Paso 4. Mida la diferencia

Compare el número de consultas antes y después, para la misma consulta y los mismos datos. Anote los dos números: son la evidencia del laboratorio.

```
# antes del cargador
$ grep -c "\[SQL" salida-antes.txt

# después del cargador, con la misma consulta
$ grep -c "\[SQL" salida-despues.txt

# también puede contarlas a ojo en la terminal del servidor
```

Si quiere generar los archivos, ejecute el servidor redirigiendo su salida: `node src/servidor.js > salida-antes.txt 2>&1`

Captura N.º 13: los dos registros del servidor lado a lado: el de antes, con una sentencia por cada venta, y el de después, con una única sentencia `WHERE venta_id IN (...)`, junto a los dos conteos. → *péguela en la Hoja de Evidencias.*

Laboratorio 6 — Errores, límites y seguridad básica

Un único endpoint que acepta consultas escritas por el cliente es cómodo y también es una superficie de ataque. Una consulta suficientemente anidada puede tumbar el servidor sin necesidad de credenciales.

Paso 1. Distinga el error del cliente del error del servidor

Un mensaje de excepción sin filtrar puede exponer la estructura de su base de datos. Agregue `formatError` al construir el servidor: el cliente recibe lo que necesita y el registro del servidor conserva el detalle completo.

```
src/servidor.js (fragmento)
const server = new ApolloServer({
  typeDefs,
  resolvers,
  formatError: (err) => {
    console.error('[GraphQL]', err);
    return {
      message: err.message,
      code: err.extensions && err.extensions.code,
      path: err.path,
    };
  },
});
```

Vuelva a ejecutar `cambiarCantidad` con cantidad 0 y compare la respuesta del explorador con lo que quedó escrito en la terminal.

Captura N.º 14: la respuesta del explorador con el error resumido y, al lado, la terminal del servidor con el detalle completo. → *péguela en la Hoja de Evidencias.*

Paso 2. Ponga un límite de profundidad

Agregue la regla de validación al servidor. Es una regla del esquema: se aplica antes de ejecutar un solo resolver.

```
src/servidor.js (fragmento)
const depthLimit = require('graphql-depth-limit');

const server = new ApolloServer({
  typeDefs,
  resolvers,
  validationRules: [depthLimit(6)],
  formatError: (err) => { /* como en el paso anterior */ },
});
```

Paso 3. Intente pasarse del límite

Escriba una consulta profunda a propósito. Como Venta tiene detalle y en su proyecto puede haber referencias circulares, un atacante encadena niveles hasta agotar el servidor.

```
query {
  venta(id: 1) {
    detalle {
      producto
    }
    cliente {
      nombre
    }
  }
}
```

ahora baje el límite a depthLimit(2) y vuelva a ejecutarla

Con el límite en 2, el servidor responde con un error de validación y no ejecuta ningún resolver. Devuelva el límite a un valor razonable para su esquema antes de continuar.

Captura N.º 15: el error de profundidad excedida devuelto por el servidor, sin que aparezca ninguna sentencia SQL en la terminal. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 7: La profundidad no es la única medida del costo de una consulta: pedir diez mil ventas con dos niveles cuesta más que pedir una venta con seis. ¿Qué otro límite agregaría usted y sobre qué lo calcularía? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Paso 4. Desactive el explorador para producción

El explorador y la introspección del esquema son excelentes para desarrollar y regalan el mapa completo de su API a cualquiera que la encuentre publicada.

```
src/servidor.js (fragmento)
const enProduccion = process.env.NODE_ENV === 'production';

const server = new ApolloServer({
  typeDefs,
  resolvers,
  introspection: !enProduccion,
  validationRules: [depthLimit(6)],
});
```

Levante el servidor con `NODE_ENV=production node src/servidor.js` e intente abrir el panel de documentación del explorador.

Laboratorio 7 — Comparar con REST midiendo, no opinando

La afirmación «GraphQL es más eficiente que REST» no vale nada sin un número al lado. Vamos a resolver la MISMA pantalla —encabezado de la venta, nombre del cliente y detalle— de las dos maneras, y a medir dos cosas: cuántos viajes de red hacen falta y cuántos bytes viajan.

Paso 1. Mida el camino REST

Con REST, la pantalla necesita tres llamadas: la venta, el cliente y el detalle. Mida cada una con curl y sume.

```
$ curl -s -o /dev/null -w "%{size_download} bytes  %{time_total}s\n" \
  http://localhost:3000/usuarios/1

# repita la medición para los otros dos recursos de la pantalla
# (la venta y su detalle, tal como los expondría su servicio REST)
```

Anote los tres tamaños y los tres tiempos. El total de viajes es 3, y el tercero no puede empezar hasta que el primero devuelva el identificador del cliente.

Captura N.º 16: las tres mediciones con curl del camino REST, con sus tamaños y tiempos. → *péguela en la Hoja de Evidencias.*

Paso 2. Mida el camino GraphQL

La misma pantalla, en un solo viaje y pidiendo exactamente los campos que se pintan.

```
$ curl -s -o /dev/null -w "%{size_download} bytes  %{time_total}s\n" \
  -H "Content-Type: application/json" \
  -d '{"query":{"{ venta(id:1){ fecha total cliente(nombre) } }"}}' \
  http://localhost:4000/

# el -d va en UNA sola línea: un salto dentro de las comillas
# simples rompe el JSON de la consulta
```

Anote el tamaño y el tiempo. Ese es un solo viaje de red desde el cliente, aunque el servidor haya hecho varios por dentro.

Captura N.º 17: la medición con curl de la consulta GraphQL equivalente, con su tamaño y su tiempo. → *péguela en la Hoja de Evidencias.*

Paso 3. Llene la tabla de comparación

Construya esta tabla con SUS números y péguela en la Hoja de Evidencias junto a la captura. No se evalúa qué opción gana: se evalúa que los números sean suyos y que la conclusión se siga de ellos.

```
informe/comparacion.md

| Medida | REST | GraphQL |
|-----|-----|-----|
| Viajes de red del cliente | | |
| Bytes descargados en total | | |
| Tiempo total observado | | |
| Campos recibidos y no usados | | |
| Consultas SQL del lado servidor | | |
```

Captura N.º 18: la tabla de comparación completada con sus propias mediciones. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 8: Con sus números a la vista: ¿en qué escenario concreto de su proyecto integrador seguiría prefiriendo REST antes que GraphQL? Justifique con al menos una de las medidas que acaba de tomar. → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Paso 4. Suba el trabajo al repositorio

Incluya el README.md explicando cómo levantar los dos servicios: el REST de la Práctica 3 y la capa GraphQL de esta práctica, en ese orden.

```
$ git init && git add .  
$ git commit -m "Práctica 4: capa GraphQL y BFF sobre el servicio REST"  
$ git remote add origin <URL-de-su-repositorio>  
$ git push -u origin main
```

Captura N.º 19: la página del repositorio en GitHub mostrando la carpeta practica4-graphql con src/, db/ y el README. → *péguela en la Hoja de Evidencias.*

PARTE 2 — EJERCICIOS PROPUESTOS

Esta parte se resuelve de forma individual. Cada ejercicio se entrega con: (a) el código o los archivos de configuración en el repositorio, (b) las capturas que demuestren su funcionamiento y (c) la explicación pedida, en el informe PDF.

Nota: Todos los ejercicios se resuelven sobre SU proyecto integrador, no sobre el sistema de ventas del laboratorio. Del 1 al 3 son la base; el 4, el 5 y el 6 son los que separan una entrega correcta de una sobresaliente.

Ejercicio 1. El esquema de su proyecto como contrato (8 pts)

Escriba el esquema GraphQL de una parte real de su proyecto integrador: al menos dos tipos relacionados en uno a muchos y las consultas que una pantalla concreta necesita.

Se pide:

- Al menos dos tipos con relación de uno a muchos, con obligatoriedad declarada campo por campo.
- Al menos tres consultas: una lista, una búsqueda por identificador y una con al menos un argumento de filtro.
- Una justificación breve, en el informe, de tres decisiones de obligatoriedad: por qué un campo lleva admiración y otro no.
- Captura del panel de documentación del explorador mostrando el esquema cargado.

Criterio de aceptación: *Un compañero debe poder escribir una consulta válida contra su esquema leyendo únicamente el panel de documentación, sin preguntarle nada a usted.*

Ejercicio 2. Resolvers con argumentos, variables y paginación (10 pts)

Implemente los resolvers de lectura de su esquema contra su base de datos, con filtros y con un límite de resultados.

Se pide:

- Los tres resolvers de consulta del ejercicio 1, funcionando contra su base real.
- Al menos una consulta con dos argumentos combinados (por ejemplo, un filtro y un límite).
- Todas las pruebas ejecutadas con variables, nunca con valores escritos dentro de la consulta.
- Un caso probado en el que la búsqueda no encuentra nada, y la respuesta que su esquema promete para ese caso.

Criterio de aceptación: *Ninguna consulta puede devolver la tabla entera sin límite: debe existir un tope, con su valor por omisión documentado.*

Ejercicio 3. Mutación con una regla de negocio que puede rechazar (10 pts)

Implemente una mutación de su proyecto que cambie el estado y que pueda negarse a hacerlo: registrar un movimiento de inventario que no deje el stock en negativo, asignar un cupo que no exceda el máximo, registrar un pago que no supere el saldo.

Se pide:

- La mutación declarada en el esquema, devolviendo el objeto resultante y no un texto de confirmación.
- La regla de negocio validada en el servidor, con el mensaje de error redactado para una persona.
- Dos capturas: la operación aceptada y la operación rechazada por la regla.
- Una verificación en la base de que la operación rechazada NO dejó datos a medias.

Criterio de aceptación: *Si la regla rechaza la operación, la base debe quedar exactamente como estaba. Una escritura parcial anula el ejercicio.*

Ejercicio 4. Backend for Frontend sobre sus propios servicios (12 pts)

Escriba un campo de su esquema cuyo valor no salga de su base de datos, sino de otro servicio: el REST de la Práctica 3, otro servicio de su proyecto integrador o un servicio público. El resultado debe llegar al cliente como un campo más, dentro de la misma respuesta.

Se pide:

- Un campo compuesto que combine datos de al menos dos orígenes distintos en un solo tipo.
- La dirección del servicio externo leída de una variable de entorno, nunca escrita fija en el código.
- El manejo del caso en que el servicio externo responde con un 404 y del caso en que no responde.
- Una captura de la respuesta parcial: data con lo que sí se resolvió y errors con lo que falló.
- Un diagrama sencillo, en el informe, que muestre qué dato viene de dónde.

Criterio de aceptación: *El cliente que consume el esquema no debe necesitar saber cuántos servicios hay detrás. Y la caída del servicio externo no puede dejar sin respuesta a los campos que no dependen de él.*

Ejercicio 5. N+1 demostrado y resuelto en su proyecto (10 pts)

Encuentre un N+1 en el esquema de su proyecto —lo habrá en cuanto pida una lista con un campo anidado— demuéstrela con registros y resuélvala con carga por lotes.

Se pide:

- Un registro de consultas que muestre las 1+N sentencias antes de corregir, con el número de sentencias visible.
- El cargador por lotes implementado y creado por petición, no una sola vez al arrancar.
- El mismo registro después de corregir, con el conteo de sentencias.
- Una prueba con al menos 20 registros en la lista, para que la diferencia sea evidente.

Criterio de aceptación: *El conteo de sentencias después de la corrección no debe crecer al aumentar el número de elementos de la lista.*

Ejercicio 6. REST contra GraphQL, medido sobre una pantalla real (10 pts)

Elija UNA pantalla real de su proyecto integrador —una que muestre datos de al menos dos entidades— y resuélvala de las dos maneras: con las llamadas REST que hoy necesita y con una sola consulta GraphQL. Después mida.

Se pide:

- La descripción de la pantalla y la lista exacta de campos que pinta, antes de medir.

- La tabla comparativa completa: viajes de red del cliente, bytes descargados, tiempo observado, campos recibidos y no usados, y consultas a la base del lado del servidor.
- Las capturas de las mediciones con curl o con la pestaña de red del navegador, en ambos caminos.
- Una conclusión de un párrafo que se siga de SUS números, incluyendo al menos un aspecto en el que REST salga mejor parado.

Criterio de aceptación: *La misma pantalla y los mismos datos en los dos caminos. Una conclusión que no se apoye en los números de su propia tabla no acredita, aunque sea correcta en general.*

Rúbrica de evaluación

Parte 1 — Práctica guiada (40 puntos)

Criterio	Descripción	Pts
Capturas completas	Las 19 capturas están pegadas en la Hoja de Evidencias, son legibles y corresponden a la máquina del estudiante (se ve su usuario o el nombre del equipo).	15
Ejecución correcta	Las salidas muestran el resultado esperado, sin errores sin resolver.	10
Preguntas de comprobación	Las 8 preguntas están respondidas con criterio propio, no copiadas del enunciado.	10
Orden y presentación	Hoja de Evidencias completa, con datos del estudiante y declaración firmada, entregada en PDF y dentro del plazo.	5
Subtotal		40

Parte 2 — Ejercicios propuestos (60 puntos)

Ejercicio	Qué se evalúa	Pts
1. El esquema de su proyecto como contrato	Tipos relacionados, obligatoriedad declarada con criterio y justificada, tres consultas útiles, esquema legible desde el explorador.	8
2. Resolvers con argumentos, variables y paginación	Resolvers funcionando contra la base propia, argumentos combinados, uso de variables, caso sin resultados resuelto según el contrato.	10
3. Mutación con una regla de negocio que puede rechazar	Mutación que devuelve el objeto, regla validada en el servidor, evidencia de aceptación y de rechazo, estado consistente tras el rechazo.	10
4. Backend for Frontend sobre sus propios servicios	Composición real de dos orígenes, dirección configurable, fallo del servicio externo manejado sin arrastrar el resto, diagrama del origen de cada dato.	12
5. N+1 demostrado y resuelto en su proyecto	N+1 demostrado con conteo, cargador por petición correctamente construido, mejora medida con volumen suficiente.	10
6. REST contra GraphQL, medido sobre una pantalla real	Pantalla real con campos declarados, tabla de cinco medidas completada, evidencia de ambas mediciones, conclusión sostenida por los datos propios.	10
Subtotal		60

Penalizaciones

Situación	Descuento
Credenciales de la base de datos escritas fijas en el código y subidas al repositorio	-5 pts
Consultas que devuelven la tabla completa sin ningún límite de resultados	-4 pts
Mutaciones que devuelven un texto de confirmación en lugar del objeto resultante	-4 pts
Reescribir el servicio REST de la Práctica 3 en lugar de componer sobre él	-5 pts
node_modules subido al repositorio (falta .gitignore)	-3 pts
Conclusiones del ejercicio 6 sin la tabla de mediciones que las respalde	-5 pts

Situación	Descuento
Capturas que no corresponden a la máquina del estudiante	Anulación de la Parte 1
Entrega fuera de plazo	Según reglamento de la materia

Anexo — Errores frecuentes y cómo resolverlos

Mensaje o síntoma	Causa probable	Solución
Cannot query field "x" on type "Venta"	El campo no está declarado en el esquema, o el servidor no se reinició después de agregarlo.	Agregue el campo al esquema y reinicie el proceso. La validación es contra el esquema cargado en memoria.
Cannot return null for non-nullable field Venta.total	El resolver devolvió undefined: el SELECT no trae esa columna o el nombre no coincide.	Revise el SELECT y use un alias AS para que el nombre de la columna coincida con el del esquema.
Unknown column "precioUnitario" in "field list"	Se usó el nombre del campo GraphQL dentro del SQL. La base sigue llamándose precio_unitario.	Traduzca en el resolver: SELECT precio_unitario AS precioUnitario.
Error: connect ECONNREFUSED 127.0.0.1:3306	MySQL no está iniciado, escucha en otro puerto o las credenciales del pool son incorrectas.	Levante MySQL y verifique host, usuario y contraseña en src/db.js antes de arrancar el servidor.
TypeError: fetch failed / ECONNREFUSED:3000	El servicio REST de la Práctica 3 no está corriendo o la dirección del entorno apunta a otro lado.	Levante el servicio de la Práctica 3 y confirme con curl antes de repetir la consulta compuesta.
La respuesta es correcta pero la terminal imprime 1+N sentencias SQL	N+1 silencioso: un resolver anidado consulta una vez por cada elemento de la lista.	Agrupe con un cargador por lotes creado por petición y consulte con WHERE ... IN (?).
Query is too deep / exceeds maximum depth	La consulta supera el límite de profundidad configurado. No es una falla: es la protección funcionando.	Reduzca el anidamiento de la consulta o ajuste el límite al valor que su esquema realmente necesita.
Respuesta 200 con "errors" y datos incompletos en "data"	Un resolver lanzó una excepción. GraphQL responde 200 y reporta el fallo campo por campo.	Lea el campo path del error para saber qué resolver falló; no se guíe por el código de estado HTTP.

Antes de pedir ayuda: Antes de pedir ayuda, responda tres preguntas: ¿el error es de validación del esquema o de ejecución de un resolver? Si es de ejecución, ¿qué dice el campo path? Y ¿cuántas sentencias SQL aparecieron en la terminal del servidor? Esos tres datos resuelven la mayoría de los casos sin ayuda de nadie.

Referencias

- Especificación de GraphQL y guía oficial — <https://graphql.org/learn/>
- Apollo Server 4, documentación oficial — <https://www.apollographql.com/docs/apollo-server/>
- DataLoader, el proyecto original de Facebook — <https://github.com/graphql/dataloader>
- Richardson, Chris. Microservices Patterns, capítulos sobre API Gateway y Backend for Frontend. Manning, 2018.
- Diapositivas del Tema 4 — Comunicación entre Microservicios (COM-600).